

● PROGRAMACIÓN AVANZADA · BACHILLERATO TECNOLÓGICO · UTU

Práctica: MiniCRUD

Tres actividades progresivas sobre el proyecto MiniCRUD: ampliar la entidad, validar datos en el servidor y construir un CRUD completo para una entidad propia.

Prof. Elizabeth Izquierdo

CONTENIDO

ACTIVIDAD 01 Ampliar la entidad Persona (teléfono + dirección)

Básico

ACTIVIDAD 02 Validar campos en el formulario de alta

Intermedio

ACTIVIDAD 03 Nuevo CRUD, entidad a elección

Avanzado

01

ACTIVIDAD 01 · NIVEL BÁSICO · PROYECTO BASE: MINI-CRUD1

Ampliar la entidad **Persona**

Agregar los campos teléfono y dirección en todas las capas del proyecto.

BÁSICO

MINI-CRUD1

5 ARCHIVOS

6 TAREAS

OBJETIVO

El proyecto `mini-crud1` ya tiene un CRUD funcional para `Persona` con los campos **código** y **nombre**. El objetivo es extenderlo para que también almacene y muestre **teléfono** y **dirección**. Los cambios deben aplicarse en el VO, en el Servlet (métodos agregar y editar) y en los tres JSP.

Archivos involucrados

- > `PersonaV0.java` Modelo · VO – nuevos campos, constructor, getters/setters
- > `PersonaControladorServlet.java` Controlador – leer 2 parámetros nuevos en `agregar()` y `editar()`
- > `persona-form.jsp` Vista · Alta – 2 inputs nuevos (telefono, direccion)
- > `persona-lista.jsp` Vista · Listado – 2 columnas nuevas en `thead` y `tbody`
- > `persona-editar.jsp` Vista · Edición – 2 campos precargados con EL

// TAREAS

- 1 `PERSONAV0.JAVA` Agregar `private String telefono` y `private String direccion`. Actualizar el constructor para que reciba 4 parámetros. Agregar getters y setters para cada campo nuevo.
- 2 `SERVLET – AGREGAR()` En el método `agregar()`, leer `request.getParameter("telefono")` y `request.getParameter("direccion")`. Pasarlos al constructor de `PersonaV0`.
- 3 `SERVLET – EDITAR()` En el método `editar()`, leer los mismos dos parámetros y construir el `PersonaV0` con los 4 campos.
- 4 `PERSONA-FORM.JSP` Agregar un `<input type="tel">` para teléfono y un `<input type="text">` para dirección, cada uno con su `<label>`. El atributo `name` debe coincidir exactamente con lo que lee el Servlet.
- 5 `PERSONA-LISTA.JSP` Agregar `<th>Teléfono</th>` y `<th>Dirección</th>` en el encabezado de la tabla. Agregar las celdas `${p.telefono}` y `${p.direccion}` dentro del `<c:forEach>`.
- 6 `PERSONA-EDITAR.JSP` Agregar los campos de teléfono y dirección con sus valores precargados usando EL: `value="${p.telefono}"` y `value="${p.direccion}"`.

// REFERENCIAS DE CÓDIGO

 PersonaV0.java – campos nuevos

```
// Atributos a agregar
private String telefono;
private String direccion;

// Constructor actualizado (4 parámetros)
public PersonaV0(int codigo, String nombre,
                 String telefono, String direccion) {
    this.codigo = codigo;
    this.nombre = nombre;
    this.telefono = telefono;
    this.direccion = direccion;
}

// Getters y setters – igual que los anteriores, pero para String
public String getTelefono() { return telefono; }
public void setTelefono(String t) { this.telefono = t; }
public String getDireccion() { return direccion; }
public void setDireccion(String d) { this.direccion = d; }
```

 PersonaControladorServlet.java – método agregar()

```
private void agregar(HttpServletRequest req, HttpServletResponse res)
    throws IOException {
    int    codigo    = Integer.parseInt(req.getParameter("codigo"));
    String nombre    = req.getParameter("nombre");
    String telefono = req.getParameter("telefono"); // ← nuevo
    String direccion = req.getParameter("direccion"); // ← nuevo
    dao.agregar(new PersonaV0(codigo, nombre, telefono, direccion));
    res.sendRedirect(req.getContextPath() + "/persona?accion=listar");
}
```

 PersonaControladorServlet.java – método editar()

```
private void editar(HttpServletRequest req, HttpServletResponse res)
    throws IOException {
    int    codigo    = Integer.parseInt(req.getParameter("codigo"));
    String nombre    = req.getParameter("nombre");
    String telefono = req.getParameter("telefono"); // ← nuevo
    String direccion = req.getParameter("direccion"); // ← nuevo
    dao.editar(new PersonaV0(codigo, nombre, telefono, direccion));
    res.sendRedirect(req.getContextPath() + "/persona?accion=listar");
}
```

● ● ● persona-form.jsp – dentro del <form>

```
<label>Nombre</label>
<input type="text" name="nombre" required />

<label>Teléfono</label>
<input type="tel" name="telefono" />

<label>Dirección</label>
<input type="text" name="direccion" />

<button type="submit" class="btn">Guardar</button>
```

● ● ● persona-lista.jsp – dos cambios

```
<!-- 1) En el <thead> -->
<th>Código</th> <th>Nombre</th>
<th>Teléfono</th> <th>Dirección</th>
<th>Acciones</th>

<!-- 2) Dentro del <c:forEach var="p"> -->
<td>${p.codigo}</td> <td>${p.nombre}</td>
<td>${p.telefono}</td> <td>${p.direccion}</td>
<td><!-- acciones --></td>
```

● ● ● persona-editar.jsp – campos precargados con EL

```
<label>Nombre</label>
<input type="text" name="nombre" value="${p.nombre}" required />

<label>Teléfono</label>
<input type="tel" name="telefono" value="${p.telefono}" />

<label>Dirección</label>
<input type="text" name="direccion" value="${p.direccion}" />

<button type="submit" class="btn">Guardar cambios</button>
```

// CRITERIOS DE VERIFICACIÓN

- ☐ El formulario de alta (persona - form . jsp) muestra los campos Teléfono y Dirección.
- ☐ Al agregar una persona, teléfono y dirección aparecen correctamente en la tabla de listado.
- ☐ El formulario de edición muestra los valores actuales de teléfono y dirección precargados.
- ☐ Al guardar la edición, los nuevos valores se reflejan en la lista.

02

ACTIVIDAD 02 · NIVEL INTERMEDIO · PROYECTO BASE: MINI-CRUD1

Validar campos en el formulario de alta

Nombre y apellido deben ser obligatorios, con manejo de errores en el servidor.

INTERMEDIO

MINI-CRUD1

2 ARCHIVOS

5 TAREAS

OBJETIVO

Agregar validación **del lado del servidor** en el método `agregar()` del Servlet: si nombre o apellido llegan vacíos, no guardar y devolver al formulario con un mensaje de error. Si los datos son válidos, guardar y redirigir (patrón **PRG**). El formulario debe mostrar el error y conservar los valores ya ingresados.

FORWARD VS. REDIRECT — DIFERENCIA CLAVE

Forward (`RequestDispatcher.forward()`): permanece en la misma petición HTTP. Los atributos puestos con `request.setAttribute()` siguen disponibles en el JSP. Se usa cuando hay error y se quiere volver al formulario con datos.

Redirect (`response.sendRedirect()`): el navegador hace una nueva petición. Los atributos del request se pierden. Se usa cuando la operación fue exitosa (patrón PRG: Post-Redirect-Get).

Archivos involucrados

- > `PersonaControladorServlet.java` Controlador — agregar validación en el método `agregar()`
- > `persona-form.jsp` Vista · Alta — mostrar error y pre-rellenar campos con atributos del request

// TAREAS

- SERVLET — VALIDAR** Dentro de `agregar()`, leer nombre y apellido con `request.getParameter()`. Verificar que ninguno sea `null` ni esté vacío (usar `.trim().isEmpty()`).
- SERVLET — FORWARD CON ERROR** Si la validación falla: poner el mensaje de error con `request.setAttribute("error", "...")`, también los valores ingresados (`request.setAttribute("nombre", nombre)` y análogo para apellido), y hacer `forward` de vuelta a `persona-form.jsp`. Usar `return` para no continuar.
- SERVLET — PRG SI VÁLIDO** Si los datos son válidos: guardar con `dao.agregar(...)` y hacer `response.sendRedirect(...)` al listado. Nunca hacer `redirect` cuando hay error (se pierden los atributos del request).
- PERSONA-FORM.JSP — MOSTRAR ERROR** Agregar un bloque condicional con JSTL: `<c:if test="${not empty error}">`. Dentro, mostrar `${error}` en un `<div>` con estilos de alerta.
- PERSONA-FORM.JSP — PRE-RELLENAR CAMPOS** Agregar `value="${nombre}"` y `value="${apellido}"` en los inputs correspondientes. Si no hubo error, esos atributos están vacíos y el campo queda vacío — comportamiento correcto.

// REFERENCIAS DE CÓDIGO

 PersonaControladorServlet.java – validar en agregar()

```
private void agregar(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    String nombre = req.getParameter("nombre");
    String apellido = req.getParameter("apellido");

    if (nombre == null || nombre.trim().isEmpty()
        || apellido == null || apellido.trim().isEmpty()) {
        req.setAttribute("error", "El nombre y el apellido son obligatorios.");
        req.setAttribute("nombre", nombre);
        req.setAttribute("apellido", apellido);
        req.getRequestDispatcher("/persona-form.jsp").forward(req, res);
        return; // ← salir sin guardar
    }

    // Datos válidos: guardar y redirigir (patrón PRG)
    dao.agregar(...);
    res.sendRedirect(req.getContextPath() + "/persona?accion=listar");
}
```

 persona-form.jsp – mostrar error y pre-rellenar

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<c:if test="${not empty error}">
    <div class="error">${error}</div>
</c:if>

<form method="post" action="${pageContext.request.contextPath}/persona">
    <input type="hidden" name="accion" value="agregar" />

    <label>Nombre</label>
    <input type="text" name="nombre" value="${nombre}" required />

    <label>Apellido</label>
    <input type="text" name="apellido" value="${apellido}" required />

    <button type="submit" class="btn">Guardar</button>
</form>
```

PARA IR MÁS LEJOS

El atributo `required` en HTML hace que el navegador valide antes de enviar el formulario, pero esa validación puede saltarse fácilmente. La validación en el Servlet **siempre debe estar**, independientemente de lo que haga el HTML. Los dos niveles se complementan: el HTML mejora la experiencia, el servidor garantiza la integridad.

// CRITERIOS DE VERIFICACIÓN

- ☐ Si se envía el formulario con el campo nombre vacío, el sistema no guarda y muestra el mensaje de error.
- ☐ Si se envía con el campo apellido vacío, el sistema no guarda y muestra el mensaje de error.
- ☐ Al volver al formulario con error, los campos ya completados conservan sus valores.
- ☐ Al enviar con datos válidos, el sistema guarda y redirige al listado sin mostrar ningún error.

03

ACTIVIDAD 03 · NIVEL AVANZADO · REFERENCIA: MINI-CRUD2

Nuevo CRUD: entidad a elección

CRUD completo con validación de unicidad de código, tomando mini-crud2 como referencia.

AVANZADO

MINI-CRUD2 (REFERENCIA) >

6 ARCHIVOS NUEVOS

7 TAREAS

OBJETIVO

Construir un CRUD completo para una entidad elegida del grid siguiente. El proyecto de referencia es `mini-crud2`, que ya incluye las 4 operaciones básicas (alta, listado, edición, baja) más validación de unicidad de código. Hay que replicar ese patrón para la entidad elegida, renombrando clases y atributos.

IMPORTANTE – NO EMPEZAR DESDE CERO

Abrir `mini-crud2` y usarlo como guía permanente. Copiar la estructura de cada archivo (VO, DAO, Servlet, JSP), cambiar el nombre de la entidad y adaptar los atributos. El patrón de navegación, el patrón PRG y la validación de unicidad ya están resueltos en `mini-crud2`.

// ELEGIR UNA ENTIDAD

Producto

- código (int)
- nombre (String)
- precio (double)
- descripción (String)

Libro

- código (int)
- título (String)
- autor (String)
- año (int)

Película

- código (int)
- título (String)
- director (String)
- año (int)

Videojuego

- código (int)
- título (String)
- género (String)
- plataforma (String)

Estudiante

- código (int)
- nombre (String)
- apellido (String)
- grupo (String)

Otra entidad

- (proponer al docente)
-
-
-

// ARCHIVOS A CREAR (EJEMPLO: PRODUCTO)

Archivos a crear – ejemplo con Producto

- > `ProductoVO.java` Modelo · VO – atributos, constructor, getters, setters
- > `ProductoDAO.java` Modelo · DAO – lista en memoria, agregar, buscarPorCodigo, editar, eliminar
- > `ProductoControladorServlet.java` Controlador – mapeo de acciones: listar, mostrarFormAgregar, agregar, mostrarFormEditar, editar, eliminar
- > `producto-form.jsp` Vista · Alta – formulario de alta con todos los campos
- > `producto-lista.jsp` Vista · Listado – tabla con todos los atributos y acciones Editar / Eliminar
- > `producto-editar.jsp` Vista · Edición – formulario con campos precargados

// TAREAS

1

Elegir la entidad del grid y anotar los 4 atributos con sus tipos de dato Java. Si se elige "Otra entidad", acordar los atributos con el docente antes de continuar.

- 2 **XXXVO.JAVA** Crear la clase VO con los 4 atributos. Escribir el constructor con todos los parámetros, los getters y setters, y un `toString()` descriptivo.
- 3 **XXXDAO.JAVA** Crear el DAO con una lista en memoria (`ArrayList`). Implementar los métodos: `listar()`, `agregar()` con retorno boolean (validar unicidad de código), `buscarPorCodigo()`, `editar()` y `eliminar()`.
- 4 **XXXCONTROLADORSERVLET.JAVA** Crear el Servlet. Mapear las acciones en `doGet()` y `doPost()`: *listar*, *mostrarFormAgregar*, *agregar*, *mostrarFormEditar*, *editar*, *eliminar*. En `agregar()`: si el DAO retorna `false`, hacer forward con error; si retorna `true`, redirect.
- 5 **XXX-FORM.JSP** Crear el formulario de alta con un campo para cada atributo. Incluir el bloque de error con JSTL (`c:if test="${not empty error}"`) y pre-rellenar campos en caso de retorno por error.
- 6 **XXX-LISTA.JSP** Crear la tabla de listado con una columna por atributo más la columna de acciones (Editar y Eliminar). Usar `<c:forEach>` y EL para recorrer la lista.
- 7 **XXX-EDITAR.JSP** Crear el formulario de edición con todos los campos precargados usando EL (`value="${obj.atributo}"`). El campo código debe ser de solo lectura (`readOnly`) para no permitir cambiar la clave.

// PATRÓN CLAVE: VALIDACIÓN DE UNICIDAD DE CÓDIGO

PersonaDAO.java (mini-crud2) – validación de unicidad en agregar()

```
public boolean agregar(PersonaVO p) {
    for (PersonaVO existente : lista) {
        if (existente.getCodigo() == p.getCodigo()) {
            return false; // ← código ya existe
        }
    }
    lista.add(p);
    return true;
}
```

PersonaControladorServlet.java (mini-crud2) – usar el boolean en agregar()

```
private void agregar(...) throws ... {
    ...
    boolean ok = dao.agregar(new PersonaVO(...));
    if (!ok) {
        req.setAttribute("error", "El código ya existe.");
        req.getRequestDispatcher(...).forward(req, res);
        return;
    }
    res.sendRedirect(...);
}
```

// CRITERIOS DE VERIFICACIÓN

- ☐ Se puede dar de alta un nuevo registro con todos los campos completos.
- ☐ Si se intenta dar de alta con un código ya existente, el sistema muestra error y no guarda.
- ☐ La tabla de listado muestra todos los registros con sus atributos correctamente.
- ☐ Se puede editar un registro existente y los cambios se reflejan en el listado.
- ☐ Se puede eliminar un registro y desaparece del listado.
- ☐ En alta y edición se aplica el patrón PRG: éxito redirige, error hace forward.